

General Purpose Registers, Sizing & The Zero-Extension Invariant

12.0 The Physics of the Register File (SRAM Multi-Porting)

Before analyzing the x86-64 register set, we must understand the physical hardware that constitutes a register. The **Register File** is not Main Memory (DRAM), nor is it Cache (standard SRAM). It is a hyper-specialized, multi-ported SRAM array sitting directly inside the Execution Core, operating at the exact frequency of the CPU (e.g., 5.0 GHz).

The Multi-Porting Dilemma

A standard L1 Cache SRAM cell has 1 Read port and 1 Write port. However, a modern Superscalar CPU might issue four instructions per clock cycle. Consider four concurrent `ADD` instructions. Each `ADD` requires two source operands and one destination operand. To execute them simultaneously, the Register File must support **8 simultaneous Reads and 4 simultaneous Writes** in a single 0.2-nanosecond clock cycle.

Building an SRAM bitcell with 12 physical access ports requires a massive amount of silicon real estate and wire routing. The physical wiring capacitance grows quadratically with the number of ports ($O(N^2)$). This physics constraint is why CPUs have historically maintained very small architectural register files (16 in x86-64, 31 in ARM64) compared to gigabytes of main memory.

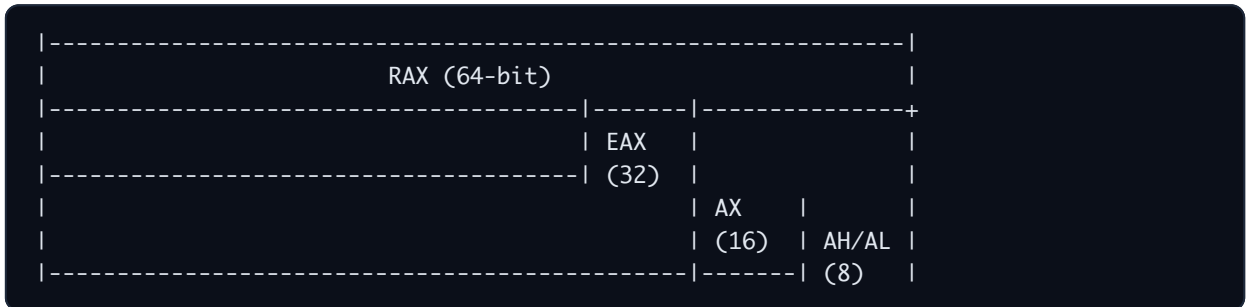
12.1 Register Evolution: The Backward-Compatibility Chains

The x86-64 architecture is a 45-year evolutionary chain of unbroken backward compatibility. The registers grew sequentially, layering on top of each other.

- **1978 (Intel 8086 - 16-Bit):** Introduced the original registers: `AX` (Accumulator), `BX` (Base), `CX` (Count), and `DX` (Data). These were 16-bit wide hardware arrays.
- **1985 (Intel 386 - 32-Bit):** Expanded to 32 bits for Protected Mode. To preserve 16-bit binaries, Intel prefixed the names with "E" (Extended): `EAX`, `EBX`, `ECX`, `EDX`. `AX` remained a valid target, mapping directly to the lower 16 bits of `EAX`.
- **2003 (AMD Opteron - 64-Bit):** AMD expanded the ISA to 64 bits (Long Mode). They prefixed the legacy registers with "R" (`RAX`, `RBX`) and introduced 8 entirely new extended registers: `R8` through `R15`.

12.2 Sub-Register Aliasing & Microarchitectural Hazards

A modern x86-64 CPU exposes 16 General Purpose Registers (GPRs). However, due to its evolutionary history, the legacy registers support fractional, overlapping sub-registers.



This overlapping structure—where `AL` (8-bit), `AX` (16-bit), `EAX` (32-bit), and `RAX` (64-bit) physically alias the exact same silicon SRAM cells—creates one of the most severe bottlenecks in Superscalar design: **The Partial Register Stall**.

The Partial Register Stall (P6 Architecture)

In Out-of-Order (OoO) execution, the CPU uses the Register Alias Table (RAT) to map architectural registers (`RAX`) to hidden Physical Registers (`P1`, `P2`). In older microarchitectures like the Intel Pentium Pro (P6), the RAT tracked dependencies at the *full 32-bit register level*.

```

; The Partial Register Hazard
mov al, 0x5      ; Write to low 8 bits (AL)
add eax, 10      ; Read the full 32-bit register (EAX)

```

When the CPU fetched `mov al, 0x5`, it updated the lower 8 bits. But when it fetched `add eax, 10`, the OoO scheduler realized that `EAX` was now fractured. The upper 24 bits belonged to the old physical register, and the lower 8 bits belonged to the new physical register. The CPU artificially halted the pipeline, invoking a microcode routine to manually execute a Read-Modify-Write (RMW) merge of the two physical registers into a single cohesive state before the `ADD` could execute. This hardware stall devastated Instruction-Level Parallelism (ILP).

The Sandy Bridge Resolution (Merging μops)

Modern CPUs (Intel Sandy Bridge onwards, AMD Zen) solved this by dynamically tracking sub-registers separately in the RAT. If the CPU detects a partial register read, it seamlessly injects a hidden "Blending μop " into the pipeline to merge the states in 1 clock cycle without a hard stall. However, compilers still aggressively avoid mixing register sizes to prevent this blending penalty.

12.3 The 32-Bit Zero-Extension Invariant

When AMD engineered the x86-64 ISA specification in 1999, they established a profound hardware invariant regarding 32-bit register operations. This rule mathematically diverges from how 16-bit and 8-bit registers behave.

The Zero-Extension Rule (AMD64 Vol. 3)

In x86-64 Long Mode, **any instruction that writes to a 32-bit sub-register (e.g., EAX, EBX, R8D) automatically zero-extends the upper 32 bits of the full 64-bit architectural register (RAX, RBX, R8).**

Consider the following x86-64 assembly execution state:

```
mov rax, 0x123456789ABCDEF0 ; RAX holds a full 64-bit value
mov eax, 5                  ; Write the value 5 into the 32-bit EAX register
```

What is the exact architectural state of RAX after the second instruction?

It is **NOT** 0x1234567800000005. Because EAX is a 32-bit target, the hardware forcefully clears the upper half of the register. The final value of RAX is strictly 0x0000000000000005.

Why Did AMD Hardwire This Rule?

In legacy 32-bit x86 architectures, compilers exclusively performed pointer arithmetic and loop counters using 32-bit registers (EAX, ECX). When migrating to 64-bit environments, pointers expanded to 64 bits. If AMD had designed the ISA such that writing to EAX left the upper 32 bits untouched (containing random dirty data from previous operations), any 32-bit math operation that was later used as a 64-bit memory pointer would crash the system with a Segmentation Fault.

The compiler would be forced to explicitly issue movsxd (Move with Sign-Extension) or bit-clearing instructions every single time it transitioned from 32-bit math to 64-bit addressing. This would heavily bloat compiled binary sizes and destroy L1 Instruction Cache density. By hardwiring 32-bit writes to auto-zero the upper half, legacy 32-bit math became instantly safe to use as 64-bit pointers without any extra software instructions.

12.4 The 16-Bit & 8-Bit Write Preservation Exception

Crucially, the zero-extension rule does NOT apply to 16-bit or 8-bit writes.

```
mov rax, 0x123456789ABCDEF0 ; RAX holds arbitrary 64-bit value
mov ax, 0x1111              ; Write into 16-bit AX register
mov al, 0x22                ; Write into 8-bit AL register
```

After executing `mov ax, 0x1111`, the upper 48 bits of `RAX` (bits 16 through 63) are **completely preserved and untouched**. The resulting value of `RAX` becomes `0x123456789ABCD1111`. Executing `mov al, 0x22` results in `0x123456789ABCD122`.

If a Low-Level C/C++ developer or reverse-engineer assumes writing to a 16-bit sub-register clears the upper bits, they will introduce silent, catastrophic mathematical corruption when those upper bits are later evaluated in a 64-bit context.

Compiler Type Casting (Sign vs. Zero Extension)

When high-level languages like C# and C++ cast narrower data types to wider data types, the JIT/AOT compiler must translate the software intent into specific hardware extensions.

```
// C# Casting Behavior
int signedInt = -5;           // 32-bit (0xFFFFFFFFB)
long signedLong = signedInt;  // 64-bit (-5, 0xFFFFFFFFFFFFFFFFB)

uint unsignedInt = 5;         // 32-bit (0x000000005)
ulong unsignedLong = unsignedInt; // 64-bit (5, 0x00000000000000005)
```

Sign-Extension (`movsxd`): To cast a signed 32-bit integer to a 64-bit integer, the compiler *must not* use the 32-bit zero-extension rule, because extending `0xFFFFFFFFB` (-5) with zeros yields `0x00000000FFFFFFFFB` (+4,294,967,291). The compiler explicitly emits `movsxd rax, eax` (Move with Sign-Extension), which copies the Most Significant Bit (the sign bit) across the entire upper 32-bit plane.

Zero-Extension (`movzx` or Implicit): To cast an unsigned 8-bit `byte` to a 32-bit `int`, the compiler uses `movzx eax, byte ptr [mem]` (Move with Zero-Extension). To cast an unsigned 32-bit `uint` to a 64-bit `ulong`, the compiler simply executes standard 32-bit math (e.g., `mov eax, ecx`), relying entirely on the AMD64 hardware invariant to zero-extend the upper bounds for free.

12.5 Assembly Optimization Idioms: Zeroing Registers

One of the most frequent operations in compiled code is initializing variables to zero. You will almost never see an optimized compiler use `mov rax, 0`. Instead, compilers exclusively emit `xor eax, eax` or `sub eax, eax`.

```
; Method A: Immediate Move to 64-bit Register
mov rax, 0          ; Opcode: 48 C7 C0 00 00 00 00 (7 bytes)

; Method B: Immediate Move to 32-bit Register
mov eax, 0          ; Opcode: B8 00 00 00 00 (5 bytes, auto-zero-extends RAX)

; Method C: The XOR Zeroing Idiom
xor eax, eax        ; Opcode: 31 C0 (2 bytes, auto-zero-extends RAX)
```

The Microarchitectural Dependency Breaker

Method C is dominant for two reasons. First, Code Density: consuming only 2 bytes drastically improves L1i Cache hit rates. Second, and far more importantly, is **Hardware Dependency Breaking**.

In an Out-of-Order execution core, instructions cannot execute until their source operands are computed by older instructions. If you execute `mov eax, 0`, the CPU scheduler must track the previous state of `EAX` to ensure accurate architectural retirement.

However, modern Intel and AMD Decoders contain dedicated **Zero-Idiom Recognizers**. When the decoder sees an instruction XORing a register against itself (`xor eax, eax`), the mathematical outcome is guaranteed to be 0, regardless of what the register previously held. The Register Alias Table (RAT) intercepts this. It assigns a fresh Physical Register initialized to 0, completely severs the dependency chain on the old `EAX`, and marks the register as *Ready* instantly. The instruction doesn't even need to be dispatched to the ALU execution ports—it is resolved entirely in the rename stage, saving massive pipeline bandwidth.

12.6 Extended Registers (R8-R15) & The REX Prefix

The AMD64 architecture provided a massive performance boost by introducing 8 new general-purpose registers: `R8` through `R15`. This doubled the register pool, allowing modern C# and C++ compilers to keep heavily used local variables entirely in the CPU core rather than "Spilling" them to the slow physical RAM stack.

The Encoding Complexity

Because the original x86 `ModR/M` byte only allocated 3 bits to identify a register, it could mathematically only address 8 registers ($2^3 = 8$). To address 16 registers, AMD had to inject an extra bit.

They achieved this by inventing the **REX Prefix** (byte values `0x40` to `0x4F`). When the compiler wants to use an extended register like `R8`, it prepends the REX byte to the instruction.

- **REX.W (Bit 3)** : Promotes the operation from 32-bit to 64-bit width.
- **REX.R (Bit 2)** : Extends the `ModR/M Reg` field to access R8-R15.
- **REX.X (Bit 1)** : Extends the `SIB Index` field to access R8-R15.
- **REX.B (Bit 0)** : Extends the `ModR/M R/M (Base)` field to access R8-R15.

The REX Byte Tax

While utilizing `R8-R15` drastically reduces memory latency by eliminating Stack Spills, it incurs a subtle **Code Density Tax**. Every single instruction that interacts with an extended register requires the 1-byte REX prefix.

```
add eax, ebx      ; 01 D8 (2 bytes)
add r8d, r9d      ; 45 01 C8 (3 bytes - REX.R and REX.B bits set)
```

For large monolithic functions, utilizing the extended registers expands the binary executable footprint by roughly 10-15%. Modern compilers like LLVM intentionally prioritize assigning variables to legacy registers (`RAX-RDI`) first, only allocating to `R8-R15` when the primary pool is exhausted, dynamically balancing Stack Spill penalties against Instruction Cache density limits.

12.7 Vector Register Evolution (XMM, YMM, ZMM)

General Purpose Registers handle scalar math (one operation per instruction). Modern high-performance software (game engines, machine learning, cryptography) relies on **SIMD (Single Instruction, Multiple Data)** execution, utilizing massive, parallel Vector Register Files.

- **SSE (Streaming SIMD Extensions):** Introduced 16 physical 128-bit registers (`XMM0` - `XMM15`).
- **AVX (Advanced Vector Extensions):** Widened the registers to 256 bits (`YMM0` - `YMM15`). The lower 128 bits map perfectly to the XMM registers.
- **AVX-512:** Widened the registers to 512 bits (`ZMM0` - `ZMM31`) and doubled the total register count to 32.

The AVX-SSE Transition Penalty & `VZERoupper`

Because the 128-bit `XMM` registers perfectly alias the lower half of the 256-bit `YMM` registers, a severe microarchitectural state hazard exists.

If a program executes a 256-bit AVX instruction (leaving dirty data in the upper 128 bits of `YMM0`) and then subsequently calls an older, legacy 128-bit SSE instruction (e.g., compiled in an external C++ DLL), the CPU enters a panic state. The hardware cannot know if the legacy SSE code expects the upper 128 bits to be preserved or discarded.

To safely switch states, the CPU forcibly halts the execution pipeline for ~70 clock cycles. It manually copies the upper 128 bits of all 16 YMM registers into a hidden internal buffer, executes the SSE instruction, and then copies them back. This **AVX Transition Penalty** completely destroys the performance benefits of vectorization.

```
// Compiler output to avoid the transition penalty
vaddps ymm0, ymm1, ymm2 ; 256-bit AVX Vector Math
vzeroupper                ; The required state-clearing instruction
call legacy_sse_function ; Safe 128-bit call
```

To prevent this massive stall, compilers aggressively inject the `vzeroupper` instruction before calling unknown external functions or returning from AVX-heavy blocks. `vzeroupper` explicitly zeroes the upper 128 bits of all YMM registers, proving to the hardware that no dirty state needs preserving, allowing an instant, 0-cycle transition to legacy SSE mode.

12.8 AVX Downclocking & Thermal Limitations

Widening registers from 64 bits to 512 bits is not just a software abstraction; it requires massive, physical 512-bit wide Arithmetic Logic Units (ALUs) executing in silicon.

When an Intel Skylake-X or Cascade Lake server CPU executes AVX-512 instructions natively across its execution ports, the transistors draw immense electrical current. The dynamic capacitance of flipping 512 bits simultaneously causes a massive thermal spike on the silicon die.

The AVX Frequency Offset

To prevent the CPU from instantly exceeding its Thermal Design Power (TDP) limit (e.g., 250 Watts) and physically melting, the hardware microcontroller intercepts AVX-512 instructions. It forces an immediate **AVX Downclocking Offset**. A CPU running at 4.0 GHz will instantaneously drop its base clock frequency to 3.0 GHz or even 2.5 GHz the moment heavy AVX-512 workloads hit the execution core.

This creates a software engineering paradox: If you vectorize your C# arrays using `Vector512<T>`, the math executes exponentially faster per cycle, but the *entire CPU frequency slows down for all threads sharing that core*. For mixed workloads (where some threads are doing scalar web routing while others are doing vector math), heavy vectorization can inadvertently cripple the scalar threads by dragging the core frequency down. Modern Cloud Native applications must heavily profile their AVX utilization to ensure the vectorization throughput actually outpaces the thermal downclocking penalty.

12.9 JIT Compiler Register Allocation Algorithms

The hardware provides 16 General Purpose Registers. However, a complex C# function might have 40 local variables. The process of mapping infinite software variables into finite hardware registers is a mathematically proven NP-Complete problem known as **Register Allocation**.

Linear Scan Allocation (Tier 0 Fast JIT)

When an application first starts, the .NET JIT or Java JVM must compile MSIL/Bytecode into assembly as fast as possible to minimize startup latency. The JIT uses a **Linear Scan** algorithm. It iterates through the variables sequentially, assigning registers until it runs out (e.g., hitting the 16th variable). Variables 17 through 40 are forcefully "Spilled" to the physical RAM stack. Linear scan is blindingly fast to compile but produces highly sub-optimal, RAM-dependent assembly.

Chaitin's Graph Coloring (Tier 1 Optimized JIT)

Once a method is identified as a "Hot Path," the Tier 1 JIT compiler re-compiles the function using advanced **Graph Coloring**.

- **Liveness Analysis:** The compiler calculates exactly when a variable is "Live" (holding data that will be read in the future). If Variable A is only used on lines 1-10, and Variable B is used on lines 12-20, their liveness intervals do not intersect.
- **Interference Graph:** The compiler builds an undirected graph. Each variable is a Node. If two variables are live at the exact same time, an Edge connects them.
- **Graph Coloring:** The compiler attempts to "color" the graph using exactly 16 colors (representing the 16 hardware registers), ensuring no two connected nodes share the same color. Because Variable A and Variable B do not intersect, they can be assigned the exact same hardware register (e.g., `R8`), dynamically reusing the silicon and preventing RAM stack spills entirely.

12.10 Chapter Verification, Checklist & Massive Exam

Verification Checklist

I can explain why SRAM multi-porting physical constraints historically limited Register File sizes.

I understand how fractional register overlapping (AL, AX, EAX) preserves backward compatibility.

I can explain Partial Register Stalls and how Sandy Bridge resolves them via blending μops .

I understand the AMD64 32-bit zero-extension rule and why it exists to protect 64-bit memory pointer bounds.

I can contrast 32-bit zero-extension against 16-bit/8-bit write preservation behavior.

I understand why `movsxd` is required for signed extension casts, preventing incorrect zero-extension.

I can explain why `xor eax, eax` breaks dependency chains in the Register Alias Table (RAT).

I understand the 1-byte REX prefix overhead required to access the extended R8-R15 registers.

I can describe the AVX-to-SSE transition state penalty and the purpose of the `vzeroupper` instruction.

I understand the thermal physics behind AVX-512 CPU downclocking offsets.

I can explain how JIT Compilers use Graph Coloring algorithms to map infinite variables into 16 hardware registers without stack spilling.

Comprehensive Examination (Staff / Principal Architecture)

1. What is the precise architectural state of `RAX` after executing `mov rax, 0xFFFFFFFFFFFFFFF` followed by `mov eax, 0x42`?

- A) `0xFFFFFFFF00000042`
- B) `0x0000000000000042` (The hardware forcefully zero-extends the upper 32 bits based on the AMD64 32-bit invariant rule).
- C) `0xFFFFFFFFFFFFFFF42`
- D) Undefined behavior due to partial register stalls.

2. Why does writing to a 16-bit sub-register like `AX` differ fundamentally from writing to `EAX` in x86-64 Long Mode?

- A) 16-bit writes clear the upper 48 bits, whereas 32-bit writes preserve them.
- B) 16-bit writes trigger hardware exceptions in Long Mode.
- C) 16-bit writes leave the upper 48 bits completely untouched (preserving existing dirty state), whereas 32-bit writes automatically zero-extend the upper 32 bits of the 64-bit register.
- D) There is no structural difference; both clear the upper bits.

3. In older microarchitectures (like Pentium Pro), executing `mov al, 0x5` followed immediately by `add eax, 10` caused a severe Partial Register Stall. What caused this stall?

- A) The 8-bit instruction required the execution of legacy 16-bit microcode.
- B) The L1 Data Cache was unable to fetch byte-sized payloads.
- C) The Out-of-Order scheduler detected that `EAX` was fractured across two different physical registers. It artificially halted the pipeline to execute a mandatory Read-Modify-Write merge before the ALU could proceed.
- D) The TLB lost the page permissions for the register memory mapping.

4. Why did AMD deliberately architect the 32-bit zero-extension rule when creating the 64-bit instruction set?

- A) It was easier to manufacture the silicon datapaths.
- B) Legacy 32-bit software exclusively used 32-bit registers for pointer arithmetic. If 32-bit writes preserved dirty data in the upper 32 bits, using those registers as 64-bit memory pointers would cause instant Segmentation Faults. Auto-zeroing allowed legacy 32-bit math to be safely evaluated as 64-bit pointers without bloating binaries with manual clearing instructions.
- C) To ensure compatibility with ARM processors.
- D) To disable AVX vector instructions during scalar operations.

5. How does the Out-of-Order execution scheduler handle the `xor eax, eax` instruction differently than a general `mov rax, 5` instruction?

- A) It bypasses the Reorder Buffer (ROB) entirely.
- B) It recognizes the XOR-with-itself idiom as completely independent of the register's prior state. It breaks the dependency chain in the Register Alias Table (RAT) and marks the register ready instantly, bypassing ALU execution entirely.
- C) It forces a 1-cycle pipeline bubble to flush sub-register aliasing.
- D) It routes the operation directly to the L1 Data Cache for permanent storage.

6. What is the microarchitectural "tax" associated with utilizing the extended `R8` through `R15` registers in a heavily optimized assembly loop?

- A) They cannot be used with SIMD instructions.
- B) Every instruction interacting with R8-R15 requires the 1-byte REX prefix. This slightly inflates binary code size, marginally reducing overall L1 Instruction Cache density.
- C) They do not support zero-extension rules.
- D) They require double the clock cycles to read from the PRF.

7. When a high-level compiler (like C#) casts a signed 32-bit `int` (-5) to a 64-bit `long`, why must it explicitly emit the `movsxd` instruction instead of relying on standard register movement?

- A) C# objects are dynamically sized at runtime.
- B) Standard register movement applies the 32-bit zero-extension rule. Zero-extending `0xFFFFFFFFB` (-5) yields `0x00000000FFFFFFFFB` (+4.2 Billion), destroying the mathematical value. `movsxd` forces true Sign-Extension, propagating the sign bit across the upper bounds.
- C) The Garbage Collector requires explicit type boundaries.
- D) Standard moves trigger the AVX-SSE transition penalty.

8. What severe pipeline hazard does the `vzeroupper` instruction actively prevent?

- A) The Partial Register Stall caused by aliasing `AL` and `RAX`.
- B) The AVX-to-SSE transition penalty. Mixing 256-bit AVX code with legacy 128-bit SSE code forces the CPU to manually save and restore the upper 128 bits of the YMM registers, causing a massive ~70-cycle hardware stall. `vzeroupper` explicitly clears the state, bypassing the stall entirely.
- C) Thermal throttling and AVX downclocking.
- D) L1 Terminal Faults (Foreshadow).

9. Why do modern server CPUs implement "AVX Downclocking offsets" when executing heavy AVX-512 workloads?

A) The L1 Cache bandwidth cannot sustain 512-bit reads at 5.0 GHz.

B) Flipping 512 bits simultaneously across massive execution ports draws immense electrical current. To prevent the silicon from exceeding its Thermal Design Power (TDP) limit and physically melting, the hardware dynamically throttles the clock frequency down (e.g., from 4.0 GHz to 3.0 GHz).

C) The Out-of-Order scheduler runs out of physical registers.

D) AVX-512 instructions require 2 cycles to decode.

10. During Tier 1 optimized JIT compilation, how does Chaitin's Graph Coloring algorithm assign 40 software variables into only 16 hardware registers without spilling to RAM?

A) It compresses multiple integers into a single SIMD vector register.

B) It calculates Liveness intervals to determine which variables are needed at the same time. Variables whose lifespans never overlap are assigned the exact same hardware register (node color), safely reusing the silicon over time without corrupting active data.

C) It uses a sequential Linear Scan algorithm.

D) It offloads the remaining 24 variables to the L1 Cache directly.

Systems Writing Prompts

Prompt 1: SRAM and Register Files

Explain the physics constraint of SRAM multi-porting. Why can modern CPUs comfortably cache megabytes of data in L2/L3 caches using single-ported cells, but are restricted to relatively small architectural Register Files (16 to 32 slots) due to the necessity of 10+ access ports per bitcell?

Prompt 2: The Invariant Application

Write out the exact hexadecimal state of the 64-bit `RBX` register before and after executing the following three sequential instructions. Clearly define when the Zero-Extension rule applies and when it does not:

```
mov rbx, 0x1111111111111111
```

```
mov ebx, 0x22222222
```

```
mov bx, 0x3333
```

Prompt 3: Partial Register Hazards

Trace the microarchitectural lifecycle of a Partial Register Stall in an older P6 pipeline. Explain the Read-Modify-Write (RMW) requirement. How did Intel Sandy Bridge mitigate this penalty by inserting hidden blending μops during the decode stage?

Prompt 4: Vector Mechanics

Explain the aliasing relationship between XMM (128-bit), YMM (256-bit), and ZMM (512-bit) registers. Detail the AVX-to-SSE transition penalty: what physically causes the ~70-cycle hardware stall, and how does the

`vzeroupper` instruction instruct the hardware state machine to bypass it?

CHAPTER 12 TECHNICAL ANSWER KEY

1. (B) Writing to a 32-bit sub-register (`EAX`) forces the hardware to automatically zero-extend the upper 32 bits of the full 64-bit architectural register (`RAX`).
2. (B) To maintain deep historical backward compatibility, 16-bit and 8-bit sub-register writes (`AX` , `AL`) exclusively modify their localized bits, preserving whatever dirty or clean data currently resides in the upper 48 bits of the register.
3. (C) The Out-of-Order CPU tracked the physical register state. Writing to `AL` assigned it to a new physical register, but `EAX` was now fractured across the new 8-bit state and the old 24-bit state. The CPU artificially stalled the pipeline to forcibly merge the components before executing the `ADD`.
4. (B) AMD recognized that if 32-bit writes preserved dirty upper data, compiled 32-bit math operations could not be safely used as 64-bit memory pointers without explicit software clearing instructions. Hardwiring the zero-extension allowed seamless, performant execution of legacy math as 64-bit pointers.
5. (B) The hardware Decoder and RAT identify the XOR-zeroing idiom. Knowing `X XOR X` is unconditionally 0, it assigns a fresh zeroed physical register, drops all prior data dependency tracking for `EAX` , and permits younger dependent instructions to execute instantly, bypassing the ALU.
6. (B) The original ModR/M byte only allocated 3 bits for register indexing ($2^3 = 8$). Expanding to 16 registers mandated the 1-byte REX prefix to provide the extra indexing bit. This 1-byte tax inflates binary instruction sizes, pushing against L1i Cache capacity.
7. (B) A negative 32-bit integer (e.g., `0xFFFFFFFFB`) must be mathematically sign-extended when widening to 64-bits (`0xFFFFFFFFFFFFFFFFB`). If the compiler used a standard move, the 32-bit zero-extension rule would trigger, yielding `0x00000000FFFFFFFFB` (+4.2 Billion), corrupting the application state.
8. (B) The hardware cannot blindly execute legacy 128-bit SSE code if the upper 128 bits of the YMM registers contain active AVX data, because the SSE specification assumes those upper bits are irrelevant. The hardware stalls to save the upper state. `vzeroupper` clears the upper bits, assuring the hardware that no state preservation is required.
9. (B) Firing 512-bit wide ALUs simultaneously generates catastrophic thermal and electrical loads. The CPU hardware strictly offsets the clock frequency (downclocking) to stay within thermal limits, which can paradoxically slow down parallel threads executing scalar math on the same core.
10. (B) Graph Coloring maps variable lifespans. Nodes represent variables; edges represent overlapping execution times. If two variables are never "Live" at the exact same clock cycle, they are assigned the same "Color" (Hardware Register), enabling massive variable arrays to multiplex gracefully within the 16 physical GPR limits without costly RAM Stack spilling.

Prompt 2 (Execution State):

Start: `0x1111111111111111`

After `mov ebx, 0x22222222` : The 32-bit rule forces zero-extension. The upper 32-bits are cleared. State:

`0x0000000022222222`

After `mov bx, 0x3333` : The 16-bit exception applies. The upper 48 bits are preserved identically. Only the lower 16 bits are overwritten. State: `0x0000000022223333`